R4.02 Cours 3

M. ZIANE

Ce cours s'inspire de videos de GeorgiaTech
https://www.udacity.com/georgia-tech

- Retour sur la couverture des branches
- Retour sur la couverture des conditions
- Les "branches" de EclEmma
- Limites des mesures de couverture
- countclumps

Couverture des branches

```
public static int sumSign (int x, int y) {
    int sum = x + y;
    int sign = 0;
    if (sum  > 0)      // C1
        sign = 1;
    else if (sum < 0)  // C2
        sign = -1;
    return sign;
}
```

- 2 branches par condition : 4 branches

Couverture des branches (suite)

```
@Test
void testSign() {
    assertEquals(1, sumSign(3, 9));  // C1 T  C2
    assertEquals(-1, sumSign(-5, -8)); // C1 F C2 T
    assertEquals(0, sumSign(-5, 5)); // C1 F C2 F
}
```


Couverture des conditions

```
public static int k (int x, int y) {
    int r;
    if (x == 0 || y >0)   // C1
        r = y / x;
    else r = y + x;
    return r;
}
assertEquals(1, k(5, 5));   // C1 T
assertEquals(0, k(5, -5));  // C1 F
```

- Ces deux tests font 100% de couverture des **branches**
- Pas pour EclEmma/Jacoco qui annonce 3 branches sur 4 !

Couverture des conditions (suite)

- La couverture de branches laisse passer la division par 0 !
- La couverture de condition exige que
  - chaque expression booléenne (et non pas décision) devienne T et F
  - sur l'ensemble des exécutions

```
assertEquals(1, k(0, -5));  // div par 0 !
```

- Ce test couvre 100% des conditions
- Mais 1 seule des 2 branches
- Il est conseillé de couvrir les **branches et les conditions**.

Les "branches" de EclEmma

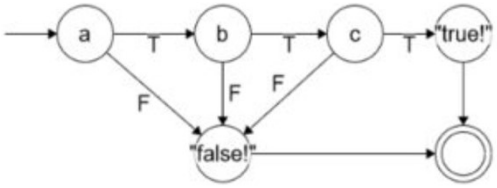
Pourquoi EclEmma affiche du jaune et compte 6 "branches" ? (ce sont plutôt des conditions)

```
public static boolean h (int x, int y, int z) {
    boolean r = false;
    if (x%2 ==0 && y%2 == 0 && z%2 == 0)
        r = true;
    return r;
}
```

```
42+ @Test
43  void testh() {
44      assertTrue(h(0, 0, 0));
45 //   assertFalse(h(0, 0, 1));
46 //   assertFalse(h(0, 1, 0));
47 //   assertFalse(h(0, 1, 1));
48 //
49 //   assertFalse(h(1, 0, 0));
50 //   assertFalse(h(1, 0, 1));
51 //   assertFalse(h(1, 1, 0));
52      assertFalse(h(1, 1, 1));
53  }
```


EclEmma

- Crédit Jf sur stackoverflow https://stackoverflow.com/questions/63492529/why-is-jacoco-coverage-report-for-branches-saying-if-a-b-c-is-actually-6
- Java évalue les opérateurs booléens de façon "paresseuse"
- Dans le bytecode les conditionnelles complexes sont dépliées



Limites des mesures de couverture

- Tests structurels et couverture de code
  - ne **remplacent pas du tout** les tests fondés sur les **spécifications**
- Des tests peuvent couvrir 100% du code mais ne pas correspondre à la spécification

```
public static int d (int x, int y) {
    int r = y  - 3;
    return x / r;
}
assertEquals(3, d(36, 15));
```

- Ce test couvre tous les chemins mais ne détecte pas la division par 0 si y vaut 3

Limites des mesures de couverture (suite)

```
public static int ur (int x) {
     int y = 0;
     int r = 0;
     if (x >5 && y >0)
          r = 1;
     return r;
}
```

Quels tests permettent de couvrir 100% des lignes ?

Limites des mesures de couverture (suite)

```
public static int ur (int x) {
     int y = 0;
     int r = 0;
     if (x >5 && y >0)
          r = 1; // non atteignable
     return r;
}
```

Quels tests permettent de couvrir 100% des lignes ?

Aucun : une ligne est non atteignable

Pourquoi du code non atteignable

- C'est parfois tout à fait normal
  - Pour une future version du logiciel
  - Pour une ancienne version du code
  - etc
- Exceptions (mais Junit peut vérifier que telle exception est bien levée)

```
assertEquals(3, div(15, 4));
assertThrows(IllegalArgumentException.class,
      () -> div(3,0));
```

- Parfois c'est un bug

CountClumps

```
public static int countClumps(int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }
        int count = 0;
        int prev = nums[0];
        boolean inClump = false;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] == prev && !inClump) {
                inClump = true;
                count += 1;
            }
            if (nums[i] != prev) {
                prev = nums[i];
                inClump = false;
            }
        }
        return count;
    }
```

Testons sans regarder la spec [Aniche 2022]

- Si on veut couvrir 100% des branches
- 3 tests suffisent
  - T1 : t est vide
  - T2 : t est nul
  - T3 : t contient un seul "clump" de 3 éléments au milieu par exemple [1, 2, 2, 2, 1]
- On peut ajouter un test pour faire 1 seul tour de boucle
  - T4 : t contient un seul élément

Très mauvais tests

- 100% des branches sont couvertes mais ce sont de très mauvais tests qui ratent de nombreux cas pertinents !
- **Il faut partir des spécifications !**
- Il manque notamment :
  - des cas avec plusieurs clumps ! (on les compte)
  - un clump à la fin du tableau
  - un clump au début
- Les tests sur le code ne sont qu'un **complément** des tests sur la spec

Tests paramétrés en Junit 5

```
@ParameterizedTest
@MethodSource("generator")
    void testClumps(int[] nums,
                    int expectedNoOfClumps) {
        assertEquals(expectedNoOfClumps,
                     Clumps.countClumps(nums));
    }
```


suite des tests de countClumps

```
static Stream<Arguments> generator() {
        return Stream.of(
            of(new int[]{}, 0),  // empty
            of(null, 0),         // null
            of(new int[]{1,2,2,2,1}, 1), // 1 clump
            of(new int[]{1}, 0), // 1 element

            // exemple de test raté
            of(new int[]{2,2}, 1)
        );
    }
```

En pratique

Suivez la démarche vue dans le cours 1

1. Comprendre les spécifications
2. Tests fondés sur les spécifications
3. Tests structurels
   - Ne pas forcément tester toutes les méthodes
     - accesseurs d'une ligne
     - code généré par un outil fiable (hashcode, equals…)
     - code trop difficile à tester
   - Couverture de lignes en premier
   - Branches **et** conditions en second
   - MC/DC seulement pour des conditions très complexes